

Grundlagen der Informatik (GDI)

Vorlesung – Hochschule Mannheim

Objektorientierung



Prof. Thomas Smits

Diese Materialien sind ausschließlich für den persönlichen Gebrauch durch Besucher der Vorlesung Grundlagen der Informatik (GDI) an der Technischen Hochschule Mannheim gedacht. Jede andere Nutzung ist ohne vorherige Zustimmung des Autors nicht zulässig.

Stand 2025-11-19

Technische Hochschule
mannheim

Inhaltsverzeichnis

1	Einführung	1
1.1	Motivation [5]	1
1.2	Objektorientierung: Definition [7]	2
1.3	Objektorientierung: Idee [9]	2
2	Klassen	4
2.1	Objekt [11]	4
2.2	Klasse [12]	5
2.3	Bestandteile einer Klasse [13]	5
2.4	Beispiel: Auto [14]	6
2.5	Erzeugung von Objekten [16]	7
2.6	Klasse vs. Objekt [18]	8
2.7	Konventionen [19]	8
2.8	Objektidentität [21]	9
2.9	Methoden [23]	9
2.10	Übergabe von Objekten an Methoden [26]	10
2.11	this-Referenz [27]	11
2.12	Beispiel: this-Referenz [29]	12
2.13	Lokale Variablen und Attribute [30]	12
2.14	Parameter und Attribute [31]	13
2.15	Konventionen [32]	13
2.16	Instanzen in UML [33]	14
3	Konstruktoren	15
3.1	Motivation [36]	15
3.2	Konstruktor [37]	15
3.3	Beispiel: Deklaration Konstruktor [39]	16
3.4	Beispiel: Aufruf des Konstruktors [40]	17
3.5	Konstruktor und this-Referenz [41]	17
3.6	Überladene Konstruktoren [42]	17
3.7	Gegenseitiger Konstruktoraufruf [43]	18
3.8	Beispiel: Euro [46]	19
3.9	Konstruktoren in UML [49]	20

4	Sichtbarkeit	21
4.1	Sichtbarkeit [51]	21
4.2	Fehlende Kapselung [52]	21
4.3	Zugriffsbeschränkung [53]	22
4.4	Beispiel mit Beschränkungen [54]	22
4.5	Zugriffsbeschränkung und UML [56]	23
5	Vererbung	25
5.1	Motivation [59]	25
5.2	The Royal Baby [62]	26
5.3	Biologische Taxonomie [65]	27
5.4	Programm-Kategorien [66]	27
5.5	Grundidee der Vererbung [67]	27
5.6	Vererbung und UML [68]	28
5.7	Begriffe [70]	29
5.8	Einfachvererbung [71]	29
5.9	Syntax [72]	30
5.10	Zurück zum Beispiel [73]	30
5.11	Generalisierung [75]	31
5.12	Spezialisierung [76]	31
5.13	Wann verwendet man Vererbung? [77]	31
5.14	Wann verwendet man Vererbung nicht? [78]	32
5.15	Vererbung und Zugriffskontrolle [79]	32
5.16	Vererbung über mehrere Ebenen [81]	33
5.17	Vererbung und Konstruktoren [82]	33
5.18	Aufrufhierarchie von Konstruktoren [83]	33
5.19	Methoden überschreiben [84]	34
5.20	Beispiel: Methoden überschreiben [85]	34
5.21	Aufruf der ursprünglichen Methode [88]	35
5.22	Überschriebene Methoden in UML [89]	36
5.23	Tipp: @Override verwenden [90]	36
5.24	Finale Klassen [91]	36
6	Object	38
6.1	Die Klasse Object [93]	38
6.2	toString-Methode [94]	38
6.3	hashCode-Methode [96]	39
6.4	equals-Methode [97]	39
6.5	Äquivalenzbeziehung [98]	40
6.6	Beziehung equals() zu hashCode() [99]	40
7	Statische Attribute und Methoden	41
7.1	Klassenattribute [101]	41
7.2	Syntax: Klassenattribute [102]	41
7.3	Beispiel: Klassenattribute [103]	41

7.4	Klassenattribute in UML [104]	42
7.5	Zugriff auf Klassenattribute [105]	42
7.6	Klassenmethoden [106]	42
7.7	Beispiel: Klassenmethode [107]	42
7.8	Klassenmethoden in UML [110]	43
7.9	Zugriff auf Klassenmethoden [111]	44
7.10	Beschränkungen in Klassenmethoden [112]	44

Kapitel 1

Einführung

1.1 Motivation [5]

- Wir kennen bisher für die Speicherung von Daten
 - ▶ *Elementare Datentypen*
 - ▶ *Arrays*
 - ▶ *Strings*
 - ▶ *Verbunde (Klassen)*
- Außerdem kennen wir *Methoden*, um auf den Daten zu operieren
- Problem: Daten und Methoden sind getrennt

```
class Mitarbeiter {  
    String name;  
    int gehalt;  
}
```



```
static void gehaltErhoehen(Mitarbeiter m, int erhoehung) {  
    m.gehalt += erhoehung;  
}
```



Bisher wurden in der Vorlesung die wichtigsten Konstrukte behandelt, die man benötigt, um Programmieraufgaben zu lösen. Es ist sogar so, dass sich jede Aufgabe mit diesen Mitteln lösen lässt.

Daten können in einfachen oder komplexen Datenstrukturen (Array, Verbunde) gespeichert werden, Methoden können auf diesen Daten operieren. Problematisch ist allerdings die Trennung von Methoden und Daten. Ständig muss man die Daten an die Methoden übergeben und entsprechende Rückgabewerte entgegennehmen und wieder in Variablen speichern. Dieses Modell entspricht aber nicht der Realität, in der wir die Dinge und die damit möglichen Operationen nicht so strikt trennen. Man denke z. B. an ein Fernsehgerät, das man einschalten

möchte. Hier ist es der natürliche Weg, einen Knopf am Fernseher selbst zu bedienen anstatt den Fernseher einer Methode `einschalten(Fernseher f)` zu übergeben.

Das Beispiel zeigt noch einmal diese unnatürliche Trennung: Um das Gehalt des Mitarbeiters um einen bestimmten Betrag zu erhöhen, muss man die Methode `gehaltErhoehen` aufrufen und einen Mitarbeiter übergeben.

Wir sind also auf der Suche nach einem natürlicheren Umgang mit Daten und Methoden, der diese Trennung aufhebt. Genau einen solchen Ansatz bietet uns die Objektorientierung, die im nächsten Abschnitt vorgestellt werden soll.

1.2 Objektorientierung: Definition [7]

Moderne Software folgt dem Prinzip der **Objektorientierung**

Objektorientierung

1. Everything is an *object*, 2. Objects communicate by sending and receiving *messages* (in terms of objects), 3. Objects have their *own memory* (in terms of objects), 4. Every object is an instance of a *class* (which must be an object), 5. The class holds the shared behavior for its instances (in the form of objects in a program list), 6. To eval a program list, control is passed to the first object and the remainder is treated as its message.

Alan Kay: The Early History of Smalltalk (1993)

- Programme bestehen aus *Objekten*
- Objekte können über *Nachrichten* miteinander kommunizieren
- Nachrichten können selber wieder Objekte enthalten
- Objekte haben *Speicher*, der wieder aus Objekten bestehen kann
- *Klassen* legen die gemeinsamen Eigenschaften von Objekten fest
- Objekte sind *Instanzen* von Klassen

Das Zitat von Alan Kay erklärt sehr gut die Grundlegende Idee der Objektorientierung. Alan Kay ist einer der Erfinder der Sprache **Smalltalk**, die als eine der ersten objektorientierten Programmiersprachen gilt. Java hat – obwohl seine Syntax stark an C++ angelehnt ist – viele Ideen von Smalltalk übernommen.

Smalltalk

Einige der Ideen von Smalltalk waren so radikal, dass sie sich von Java übernommen wurden; vor allem, um die Laufzeitgeschwindigkeit von Java-Programmen nicht negativ zu beeinflussen. So hat sich Java dagegen entschieden, Klassen selber wieder als Objekt zu realisieren. Die restlichen Punkte aus Kays Definition finden sich aber in Java und fast allen objektorientierten Programmiersprachen wieder.

1.3 Objektorientierung: Idee [9]

Objektorientierung versucht, möglichst nah an der realen Welt zu bleiben

- Die Welt besteht aus Objekten (z. B. Autos)
- Objekte haben *Eigenschaften* (z. B. Farbe des Autos)
- Objekte haben *Fähigkeiten* (z. B. fahren)

- Objekte haben *Beziehungen* zu anderen Objekten (z. B. die Räder des Autos)
- Objekte können miteinander *kommunizieren* (z. B. Motor treibt die Räder an)

Keine Trennung zwischen Eigenschaften (*Daten*) und Fähigkeiten (*Methoden*)

Objektorientierung hebt die Trennung zwischen Daten und Methoden auf und bringt die Methoden zu den Daten (oder die Daten zu den Methoden). Hierbei nimmt der Begriff **Objekt** eine zentrale Rolle ein. Das Objekt trägt sowohl die Daten (**Eigenschaften**) als auch die **Fähigkeiten**, repräsentiert durch die Methoden, die man auf dem Objekt aufrufen kann. Objekte sind außerdem in der Lage Beziehungen zu anderen Objekten zu haben, d. h. andere Objekte zu kennen und mit ihnen zu kommunizieren.

Objekt
Eigenschaften
Fähigkeiten

Die Kommunikation zwischen Objekten läuft hier so ab, dass ein Objekt auf einem anderen Objekt eine Methode aufruft. In der Objektorientierung spricht man beim Aufrufen von Methoden auch oft von **Nachrichten**, die ein Objekt dem anderen sendet.

Nachrichten

Ausgehend von einem ersten Objekt und einem ersten Methodenaufruf, entsteht der gesamte Programmablauf durch das Aufrufen von Methoden (versenden von Nachrichten). Den ersten Aufruf muss man von außen initiieren, der restliche Ablauf entsteht dann wie bei einer Kette von Domino-Steinen von alleine, da sich die Objekte gegenseitig Nachrichten senden.

So kompliziert dieses Modell auf den ersten Blick erscheinen mag, so einfach ist es in der Realität. Auch unsere bisherigen Programme haben nach diesem Prinzip funktioniert. Von außen wurde die *main*-Methode gerufen, die alle weiteren Methoden aufruft und so den Programmfluss erzeugt.

Kapitel 2

Klassen

2.1 Objekt [11]

- Ein **Objekt** (object) repräsentiert ein einzelnes, konkretes Exemplar
 - ▶ Auto mit dem Kennzeichen UN-IX 1970
 - ▶ Brad Pitt
 - ▶ BluRay mit dem Film „Pulp Fiction“ in meinem Regal
 - ▶ Mein Laptop mit der MAC-Adresse A0:FF:E3:09:6E:34
- Objekte lassen sich Klassen zuordnen
 - ▶ Fahrzeuge der Marke Renault, Modell Clio
 - ▶ Schauspieler
 - ▶ BluRays des Film „Pulp Fiction“
 - ▶ MacBook Pro

Objekt

Für das Verständnis der Objektorientierten Programmierung ist der Unterschied zwischen *Objekt* und *Klasse* entscheidend. Das Objekt repräsentiert ein einzelnes Exemplar einer bestimmten Art von Objekten. Die Klasse hingegen fasst die Gemeinsamkeiten aller Objekte zusammen, die sich unter einem Oberbegriff zusammenfassen lassen.

So ist z. B. Brad Pitt ein konkretes Objekt, denn es gibt – soweit wir wissen – nur einen einzigen Menschen mit diesem Namen, der als Schauspieler weltbekannt ist und so aussieht wie Tyler Durden aus dem Film „Fight Club“. Es mag zwar noch andere Menschen geben, die „Brad Pitt“ heißen, aber es handelt sich dann um andere Personen und nicht den Schauspieler, den wir normalerweise mit dem Namen verbinden. Der Name ist also hier nicht eindeutig, das Objekt (die Person) aber schon.

Neben Brad Pitt gibt es noch eine Reihe von weiteren Schauspielern mit Weltruhm, z. B. George Clooney. Wir wissen sicher, dass George Clooney und Brad Pitt unterschiedliche Menschen sind und beide teilweise in denselben Filmen auftreten, z. B. in „Ocean’s Eleven“. Insofern sind Brad Pitt und George Clooney unterschiedliche Objekte obwohl sie viele Gemeinsamkeiten haben. Trotz ihrer Unterschiede haben sie aber so viele Gemeinsamkeiten, dass wir uns erlauben sie beide mit Begriffen wie „Mensch“, „Schauspieler“, „Superstar“ etc. zu bezeichnen.

2.2 Klasse [12]

- **Klasse** (class) beschreibt den Typ von Objekten (führen neuen *Datentyp* ein)
- Bauplan (Schablone) für gleichartige Objekte
 - ▶ Konstruktionszeichnung für einen Auto
 - ▶ Tierrasse in der Biologie
- Fasst damit gleichartige Objekte zusammen
 - ▶ gleichen Eigenschaften (Attributen)
 - ▶ gleichem Verhalten (Methoden)
- Attribute und Methoden bilden eine Einheit

Klasse

Eine Klasse fasst nun die Gemeinsamkeiten von Objekten zusammen. So wie wir Brad Pitt und George Clooney mit dem Begriff „Schauspieler“ bezeichnen können, fasst die Klasse **Schauspieler** die gemeinsamen Eigenschaften von beiden (und allen anderen Schauspielern) zusammen. Ein Schauspieler zeichnet sich durch einen Künstlernamen, eine Liste von Filmen in denen er mitgespielt hat etc. aus.

In der Programmierung dient eine Klasse dazu, Objekte über deren Gemeinsamkeiten zu beschreiben. Anstatt also jedes Objekt einzeln detailliert zu definieren, erzeugt man mit der Klasse eine Schablone, mit der neue Objekte erzeugt werden können.

Man kann sich eine Klasse wie einen Stempel vorstellen und die Objekte als die Stempelabdrücke. Es gibt nur einen Stempel (die Klasse) aber beliebig viele Abdrücke (die Objekte).

In der Programmierung werden in einer Klasse nicht nur die Eigenschaften beschrieben, die wir **Attribute** nennen, sondern auch das Verhalten der Objekte. Unter Verhalten verstehen wir die Aktionen, die Objekte ausführen können und die wir mit den Objekten durchführen können. Das Verhalten manifestiert sich in Form von **Methoden**, die das Objekt trägt und die auf den Attributen des Objekts operieren.

Attribute**Methoden**

2.3 Bestandteile einer Klasse [13]

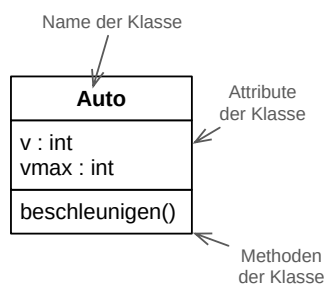
- **Name**, z. B. **Auto**
 - ▶ immer im Singular
 - ▶ großer Anfangsbuchstabe
- Eigenschaften (**Attribute**), z. B. **vmax**
 - ▶ Daten, die alle Objekte der Klasse tragen sollen
 - ▶ kleiner Anfangsbuchstabe
- Konstruktoren (werden später erst eingeführt)
- Verhalten (**Methoden**), z. B. **beschleunigen**
 - ▶ typische Verhaltensweisen für alle Objekte der Klasse
 - ▶ kleiner Anfangsbuchstabe

Wir haben die Klasse bereits kennengelernt, als der *Verbund* eingeführt wurde. Die Nutzung der Klasse als reiner Datenspeicher ist aber nur ein erster Schritt. Viel wichtiger ist die Tatsache, dass die Klassen neben den eigentlichen Daten (*Attribute*) auch das Verhalten der Objekte beschreiben, also *Methoden* enthalten.

Später werden noch die *Konstruktoren* behandelt, die bei der Erzeugung von Objekten einer Klasse eine entscheidende Rolle spielen.

2.4 Beispiel: Auto [14]

```
class Auto {  
    /** Maximalgeschwindigkeit in km/h */  
    int vmax;  
  
    /** Aktuelle Geschwindigkeit in km/h */  
    int v;  
  
    /**  
     * Beschleunigt das Fahrzeug um 1 km/h, aber  
     * höchstens bis zu Maximalgeschwindigkeit  
     */  
    void beschleunigen() {  
        if (v <= vmax) {  
            v++;  
        }  
    }  
}
```



UML-Diagramm einer Klasse

Das Beispiel zeigt eine Klasse, die neben den Attributen (*vmax* und *v*) auch eine Methode enthält. Die Methode operiert direkt auf den Attributen des Objektes. Ruft man also die Methode *beschleunigen()* auf, wird der Wert des Attributes *v* erhöht. Es ist nicht nötig der Methode Daten zu übergeben, da sie weiß, auf welchem Objekt sie arbeiten soll.

Später wird noch ein Beispiel zum Verhalten der Methode gegeben.

2.5 Erzeugung von Objekten [16]

Instanziierung bezeichnet das Anlegen eines neuen Objektes

Instanziierung

- Objekte werden mit dem **new**-Operator erzeugt (vgl. Kapitel zu Verbund)
- Syntax: **new** DATENTYP()
- Objekt hat genau die Attribute, die in der Klasse festgelegt wurden

```
Auto porsche = new Auto();
porsche.vmax = 289;

System.out.println(porsche.v); // -> 0
System.out.println(porsche.vmax); // -> 289
```



Da eine Klasse nur als Bauplan für Objekte dient, muss man das Objekt aus der Klasse mit **new** erzeugen. Bei **new** handelt es sich um einen Operator, der den Namen einer Klasse, gefolgt von () bekommt und ein neues Objekt der Klasse zurückliefert.

Wie bereits in einem früheren Kapitel erläutert handelt es sich bei allen Datentypen, deren Objekte mit **new** erzeugt werden um *Referenzdatentypen*. Insofern ist im Beispiel die Variable **porsche** eine Referenzvariable, die auf ein Objekt vom Typ **Auto** zeigt.

Das Attribut **v** des Objektes wurde nicht mit einem Wert belegt insofern ist sie mit dem sogenannten **Default-Wert** belegt, hier einer 0, weil es sich um eine Ganzzahl handelt. Für Fließkommazahlen ist der Default-Wert 0.0, für boolesche Variablen **false** und für Referenzdatentypen **null**.

Default-Wert

- Aus einer Klasse sind beliebig viele Objekte instanzierbar
- Auch als **Instanzen** (**instances**) dieser Klasse bezeichnet
- Jede Instanz belegt Speicher für die Attributwerte des Objektes

Instanzen

Klassenextension

Klassenextension

- Gesamtheit aller aktuell vorhandenen Instanzen dieser Klasse
- muss aufgrund endlichen Hauptspeichers ebenfalls endlich sein
- nicht mehr benötigte Objekte werden entsorgt, damit wieder Platz für neue entsteht

Da die Klasse nur ein Bauplan darstellt, kann man mit **new** beliebige viele Objekte dieser Klasse erstellen – die einzige Begrenzung ist der Hauptspeicher des Computers. Die Klasse selbst gibt es aber nur ein einziges Mal, so wie auch der Bauplan eines Autos nur einmal existiert, auch wenn man tausende Fahrzeuge baut.

Da jedes Objekt andere Daten tragen kann, wird für jedes Objekt im Hauptspeicher Platz reserviert. Wie viel Platz benötigt wird, ergibt sich aus der Klasse. Im Objekt können diese Attribute dann mit Werten belegt werden.

2.6 Klasse vs. Objekt [18]

Klasse legt fest

- *Typ* des Objekts
- *Attribute* (nicht jedoch deren konkrete Attributwerte)
- *Methoden*, d. h. das Verhalten des Objekts

Objekt legt fest

- *Objektidentität* (eindeutige ID oder Speicheradresse)
- konkrete *Attributwerte*, d. h. Zustand des Objekts

Zum besseren Verständnis soll noch einmal die Abgrenzung Objekt zu Klasse ausgeführt werden. Eine *Klasse* beschreibt, welche Attribute ein Objekt haben kann, z. B. bei einem Schauspieler *name* und *geburtsdatum*. Das *Objekt* enthält dann, den konkreten Wert, z. B. bei einem Schauspieler *Brad Pitt* und 18.12.1963.

Alle Objekte einer Klasse haben denselben *Typ*, der von ihrer Klasse bestimmt wird. Der Begriff des Typs ist ausgesprochen wichtig in der Programmierung. Jede Variable, jedes Literal und jeder Ausdruck hat einen Typ. Objekte als komplexe Datentypen haben ebenso einen Typ, wie Variablen von einem primitiven Datentyp (z. B. *int* oder *double*). In Java müssen die Typen bei Zuweisungen mit = zusammenpassen, sonst wird das Programm nicht kompiliert.

Typ

2.7 Konventionen [19]

- Klassenname
 - ▶ beginnt mit Großbuchstabe
 - ▶ danach gemischte Groß-/Kleinschreibung
 - ▶ kein Name, den Java bereits für eine mitgelieferte Klasse verwendet (z. B. *String*, *StringBuilder*, *Scanner*, *Math*)
- Pro Klasse eine Quellcodedatei
 - ▶ Dateiname: *Klassenname.java*
 - ▶ Kompilierter Code: *Klassenname.class*
- Reihenfolge in der Quellcodedatei
 - ▶ Attribute
 - ▶ Konstruktoren
 - ▶ Methoden

Die Namenskonventionen dienen hauptsächlich dazu, die Übersicht zu behalten. In komplexen Konstrukten ist es sinnvoll, wenn man bereits durch den Großbuchstaben im Namen erkennt, dass es sich um eine Klasse und nicht um eine Variable handelt.

Die Reihenfolge in der Quellcodedatei soll den logischen Aufbau der Klasse widerspiegeln. Sie ist nicht zwingend, da in Java Methoden auch auf Attribute und andere Methoden zugreifen

können, die nach ihnen in der Quellcodedatei deklariert wurden. Es bietet sich aber an zuerst die Attribute zu definieren, da sie die Daten der Objekte beschreiben, dann folgen die Konstruktoren (s. u.), mit denen man die Objekte erzeugen kann und dann die Methoden.

2.8 Objektidentität [21]

Jeder Aufruf von `new` erzeugt neues Objekt mit eigener **Identität** (identity)

Identität

- In Java bekommt jedes Objekt bei der Erzeugung eine eindeutige Seriennummer, die es sein gesamtes Leben lang behält
- Seriennummer kann mit `System.identityHashCode(ref)` ausgelesen werden (wird selten benutzt)

Zwei Referenzvariablen können auf ein Objekt zeigen

- Mit `==` Operator kann geprüft werden, ob Referenzvariablen auf dasselbe Objekt zeigen

```
Auto porsche = new Auto();
Auto m6 = new Auto();
Auto bmw = m6;

int idPorsche = System.identityHashCode(porsche);
int idM6 = System.identityHashCode(m6);
int idBMW = System.identityHashCode(bmw);

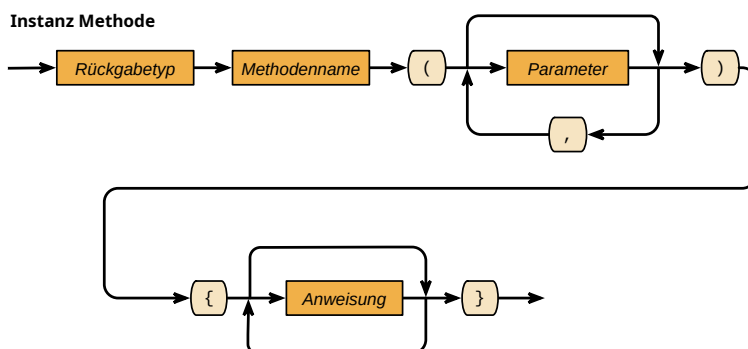
System.out.println(idPorsche); // -> 634608626
System.out.println(idM6); // -> 1463386180
System.out.println(idBMW); // -> 1463386180

System.out.println(porsche == m6); // -> false
System.out.println(bmw == m6); // -> true
```



2.9 Methoden [23]

Instanzmethoden (instance methods) sind Methoden, die auf den Daten des Objektes arbeiten

Instanzmethoden

Syntax der Deklaration einer Instanzmethode

```
Auto porsche = new Auto();
porsche.vmax = 289;
porsche.beschleunigen();
porsche.beschleunigen();

Auto m6 = new Auto();
m6.vmax = 305;

System.out.println("Porsche: " + porsche.v + " km/h");
System.out.println("M6: " + m6.v + " km/h");
```

Ausgabe

```
Porsche: 2 km/h
M6: 0 km/h
```

Das wichtigste an Klassen sind die *Instanzmethoden*. Diese Methoden arbeiten auf den Daten des Objektes. Dies bedeutet insbesondere, dass die Methode bei ihrem Aufruf zusätzlich zu den übergebenen Methoden-Parametern noch Zugriff auf die Attribute des Objekts hat. Die Methode kennt also das Objekt „auf dem sie aufgerufen wurde“. Das Aufrufen der Methode „auf dem Objekt“ wird dadurch gekennzeichnet, dass man den Punktoperator mit einer Referenzvariable `.` benutzt.

Einen Ausdruck der Form `referenz.methode()` muss man wie folgt lesen: Rufe die Methode `methode()` auf dem Objekt auf, auf das `referenz` zeigt.

2.10 Übergabe von Objekten an Methoden [26]

```
class Wand {
    void crash(Auto auto) {
        auto.v = 0;
    }
}
```

```
Wand wand = new Wand();
Auto porsche = new Porsche();
porsche.vmax = 289;
porsche.v = 165;

System.out.println(porsche.v); // -> 165

wand.crash(porsche);

System.out.println(porsche.v); // -> 0
```

Man kann Objekte auch an Methoden übergeben. Hierzu übergibt man eine Referenzvariable, die auf das Objekt zeigt und die Methode greift über die Referenz auf das übergebene Objekt zu.

Im Beispiel gibt es eine Klasse `Wand` mit einer Methode `crash`. Wenn man diese Methode mit einem Auto aufruft, wird die Geschwindigkeit des Fahrzeugs auf 0 gesetzt. Dies geschieht, indem man eine Referenz auf das Auto übergibt und die Methode diese Referenz nutzt, um die Geschwindigkeit im Auto auf 0 zu setzen.

2.11 this-Referenz [27]

- In jeder Methode gibt es eine spezielle Referenzvariable, die auf das eigene Objekt zeigt, die **this-Referenz**
- Über die **this-Referenz** greift die Methode auf Attribute des Objektes zu
- Sie verhält sich ähnlich einer normale Referenzvariable (allerdings keine Zuweisung möglich)
- Wenn in einer Methode ein Attribut oder eine andere Methode verwendet wird, setzt der Compiler automatisch `this` davor

this-Referenz

Damit die Methode Zugriff auf die Daten des Objektes bekommt, auf dem sie gerufen wurde, benötigt sie eine Referenz auf dieses. Dies wird in Java so realisiert, dass in jeder Instanz-Methode einer Referenzvariable mit dem Namen `this` vorhanden ist, die auf das Objekt verweist, auf dem die Methode aufgerufen wurde.

Wenn eine Methode auf eine Instanzvariable zugreift, erfolgt dies immer über die `this`-Referenz, auch, wenn man das auf den ersten Blick nicht sieht. Es ist nämlich in Java erlaubt, das `this` wegzulassen, wenn der Name des Attributes eindeutig ist, es also keine andere **lokale** Variable mit dem selben Namen gibt. Das folgende Beispiel verdeutlicht dies.

Zugriff auf Attribute ohne this

```
void beschleunigen() {  
    if (v <= vmax) {  
        v++;  
    }  
}
```



Zugriff auf Attribute mit this

```
void beschleunigen() {  
    if (this.v <= this.vmax) {  
        this.v++;  
    }  
}
```



Im Beispiel ist in der ersten Variante das `this` weggelassen worden, weil es keine lokale Variable `v` gibt. Somit ist klar, dass sich `v++` auf das Attribut bezieht. Der Compiler schreibt in diesem Fall das `this` automatisch vor die Variabel, sodass sich zur Laufzeit die zweite Variante (`this.v++`) ergibt.

2.12 Beispiel: this-Referenz [29]

```
class Auto {
    int vmax;
    int v;

    /**
     * Vergleicht das andere Auto mit diesem.
     * @param auto das andere Fahrzeug
     * @return true bei gleichem Inhalt, sonst false
     */
    boolean gleich(Auto auto) {
        if (auto == this) {
            return true;
        }
        else {
            return (this.vmax == auto.vmax) && (this.v == auto.v);
        }
    }
}
```

Das Beispiel zeigt noch einmal, dass eine Methode sowohl auf den Daten des eigenen Objekts als auch den Daten eines fremden Objekts arbeiten kann. Bei einem fremden Objekt, muss ihr dieses aber explizit übergeben werden.

Die Methode `gleich` soll das aktuelle Objekt mit einem übergebenen Objekt vergleichen. Hierzu bekommt sie das andere Objekt übergeben.

Im ersten Schritt (`if (auto == this)`) wird überprüft, ob das übergebene Objekt zufällig das eigene ist. Dies könnte passieren, wenn der Aufrufer z. B. schreiben würde `Auto a ← = new Auto(); a.gleich(a)`. Das in diesem Fall unweigerlich `true` herauskommen muss sollte offensichtlich sein.

Im zweiten Schritt werden die Daten aus dem übergebenen Objekt mit denen aus dem eigenen verglichen. Die eigenen Attribute sind über `this` erreichbar, die fremden über die entsprechende Referenz `auto`.

2.13 Lokale Variablen und Attribute [30]

- Lokale Variablen **verdecken** (shadow) gleichnamige Attribute
- Zugriff auf die Attribute kann dann über `this` erfolgen

verdecken

```
class Auto {
    int vmax;

    void methode() {
        int vmax; // überdeckt Attribut vmax
        vmax = 200; // Zugriff auf lokale Variable
        this.vmax = 120; // Zugriff auf Attribut
    }
}
```

Man kann das `this` nicht mehr weglassen, wenn der Name doppeldeutig ist. Im vorliegenden Beispiel ist das die Variable `vmax`, die sowohl als Attribut als auch als lokale Variable existiert. Hierdurch ist unklar, wer mit `vmax` gemeint ist.

Um dieses Dilemma aufzulösen hat sich Java entschieden, der lokalen Variable den Vorrang zu geben. D. h. schreibt man im Beispiel `vmax`, ist damit die lokale Variable gemeint. Das Attribut ist dann nicht mehr unter diesem Namen erreichbar – man sagt auch, dass „die lokale Variable das Attribut *verdeckt*“. Will man trotzdem noch auf das Attribut zugreifen, muss man zwingend `this` davor schreiben.

2.14 Parameter und Attribute [31]

- Parameter **verdecken** (shadow) gleichnamige Attribute
- Zugriff auf die Parameter kann dann über `this` erfolgen

verdecken

```
class Auto {
    int vmax;

    void methode(int vmax) { // überdeckt Attribut vmax
        System.out.println(vmax); // Zugriff auf Parameter
        System.out.println(this.vmax); // Zugriff auf Attribut
    }
}
```



Da Parameter nichts anderes sind als lokale Variablen, können auch sie Attribute verdecken. Hier gilt, analog zu den vorherigen Ausführungen, dass der Zugriff auf das Attribut in diesem Fall über `this` erfolgen muss.

2.15 Konventionen [32]

Konvention zur Benennung von Parametern und Variablen

- Lokale Variablen sollten *nicht* so heißen, wie Attribute
- Parameter sollten *nicht* so heißen, wie Attribute

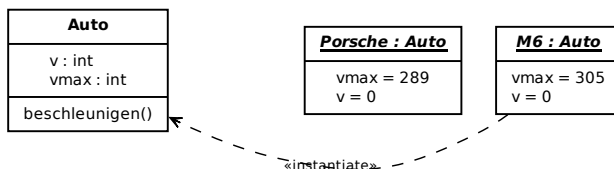
Ausnahmen

- Konstruktoren (siehe unten)
- Setter (siehe unten)

Da das Verdecken von Attributen durch lokale Variablen unangenehm ist und zu verwirrenden Ergebnissen führen kann, gilt es gemeinhin als schlechter Stil. Deshalb sollte man Parameter und lokale Variablen immer anders benennen als die Attribute.

Von dieser Regel wird häufig bei Konstruktoren und sogenannten Settern abgewichen, da hier gleichnamige Variablen sogar die Verständlichkeit des Programms erhöhen können. Konstruktoren und Setter werden später noch behandelt.

2.16 Instanzen in UML [33]



Objektbenennung (immer vollständig unterstrichen)

- Objektname, z. B. ThomasPorsche911
 - ▶ Objekt bekommt Name wie in realer Welt
 - ▶ kann zugleich Variablenname sein
- Objektname : Klassenname, z. B. Porsche : Auto
 - ▶ zusätzlich zum Objektnamen folgt nach dem : der Klassenname
 - ▶ sinnvoll, wenn «instantiated»-Beziehung im Diagramm fehlt
- :Klassenname, z.B. :Auto
 - ▶ nur : und danach der Klassenname
 - ▶ für anonyme (namenlose) Objekte

Es gibt in UML die Möglichkeit neben Klassen auch Instanzen darzustellen. Hierzu wird eine Notation verwendet, die ähnlich zu den Klassen ist. Für die Attribute können die Werte angegeben werden. Die Beziehung zur Klasse wird durch einen Pfeil mit dem Text «instantiated» dargestellt.

Kapitel 3

Konstrukturen

3.1 Motivation [36]

Häufig möchte man bei der Erzeugung eines Objektes die Attribute des Objektes initialisieren

```
class Kfz {  
    int leistung;  
    int vmax;  
    double gewicht;  
}
```



```
Kfz porsche = new Kfz();  
porsche.leistung = 350;  
porsche.vmax = 289;  
porsche.gewicht = 1.455;
```



```
Kfz m6 = new Kfz();  
m6.leistung = 507;  
m6.gewicht = 1.785;
```

Bisher haben wir Objekte immer leer angelegt und dann schrittweise über die Attribute mit Daten gefüllt. Wie man im obigen Beispiel deutlich sehen kann ist dieses Vorgehen durchaus umständlich und langwierig. Außerdem kann man leicht ein Attribut vergessen, so wie das Attribut `vmax` beim zweiten Objekt im Beispiel.

Elegant wäre eine Lösung bei der man die Objekte direkt bei der Erzeugung mit `new` mit den richtigen Werten initialisieren könnte.

3.2 Konstruktor [37]

Konstruktor (constructor)

Konstruktor

- dient der Initialisierung von Objekten bei deren Erzeugung
- verhält sich *ähnlich* einer Methode

- wird automatisch bei der Objekterzeugung durch `new` aufgerufen
- kann Parameter haben und verwenden, z. B. `Scanner scanner = new Scanner(System.in);`
- kann überladen werden für unterschiedliche Initialisierungen

Der Konstruktor

- heißt genauso wie die Klasse
- hat keinen Rückgabetyt
- nicht mit Methode verwechseln
- Syntax: `KLASSENNAME(PARAMETER){ ... }`

Wenn kein Konstruktor definiert wird erzeugt der Compiler einen parameterlosen Standard-konstruktor (**Default Constructor**)

Default Constructor

Die Lösung für die Initialisierung von Objekten sind die *Konstruktoren*. Sie verhalten sich ähnlich wie Methoden, werden aber bei der Objekterzeugung mit `new` aufgerufen. Da man ihnen Parameter übergeben kann, besteht die Möglichkeit über den Konstruktor die Attribute des Objektes zu setzen.

3.3 Beispiel: Deklaration Konstruktor [39]

```
class Kfz {  
    int leistung;  
    int vmax;  
    double gewicht;  
  
    /**  
     * Neues Fahrzeug anlegen.  
     * @param leist Leistung in PS  
     * @param vmx Maximalgeschwindigkeit in km/h  
     * @param gew gewicht in Tonnen  
     */  
    Kfz(int leist, int vmx, double gew) {  
        leistung = leist;  
        vmax = vmx;  
        gewicht = gew;  
    }  
}
```



Das Beispiel zeigt einen Konstruktor für die Klasse `Kfz`. Der Konstruktor hat die Parameter, die nötig sind, um das Objekt korrekt mit Daten zu befüllen. Innerhalb des Konstruktors werden die Attribute dann mit den übergebenen Werte initialisiert.

3.4 Beispiel: Aufruf des Konstruktors [40]

Aktualparameter werden an den Konstruktor beim `new` übergeben

```
Kfz porsche = new Kfz(350, 289, 1.455);  
Kfz m6 = new Kfz(507, 305, 1.785);  
Kfz smart = new Kfz(45, 160, 0.805);
```



Der Konstruktor wird beim `new` automatisch aufgerufen. Die Parameter für den Konstruktor schreibt man einfach in die Klammern nach dem Klassennamen. Insofern kann man sich den rechten Operanden des `new`-Operators auch einfach als dem Aufruf des Konstruktors mit seinen Parametern vorstellen. Hierbei müssen Anzahl und Typ der übergebenen Werte mit denen des Konstruktors übereinstimmen.

3.5 Konstruktor und this-Referenz [41]

- Auch der Konstruktor hat eine `this`-Referenz
- Konvention: Parameter des Konstruktors heißen so wie die Attribute

```
class Kfz {  
    int leistung;  
    int vmax;  
    double gewicht;  
  
    Kfz(int leistung, int vmax, double gewicht) {  
        this.leistung = leistung;  
        this.vmax = vmax;  
        this.gewicht = gewicht;  
    }  
}
```



Der Konstruktor dient der Initialisierung des Objektes. Da zuerst der Speicher für das Objekt angelegt wird (mit Default-Werten für die Attribute) und dann der Konstruktor gerufen wird, hat der Konstruktor eine `this`-Referenz, die auf das gerade frisch erzeugte Objekt zeigt. Mit Hilfe dieser Referenz kann der Konstruktor auf die Attribute des Objektes zugreifen und ihnen Werte zuweisen.

3.6 Überladene Konstruktoren [42]

Konstruktoren können (ebenso wie Methoden) überladen werden

```
class Kfz {  
    int leistung;  
    int vmax;
```



```
double gewicht;

Kfz(int leistung, int vmax, double gewicht) {
    this.leistung = leistung;
    this.vmax = vmax;
    this.gewicht = gewicht;
}

Kfz(int leistung, int vmax) {
    this.leistung = leistung;
    this.vmax = vmax;
    this.gewicht = 1.5;
}
}
```

Konstruktoren können, ebenso wie Methoden, überladen werden. D.h. eine Klasse kann beliebig viele Konstruktoren haben, solange sich die Parameter in Anzahl und/oder Typ unterscheiden. (Siehe hierzu auch noch einmal die Ausführungen zu überladenen Methoden.)

3.7 Gegenseitiger Konstruktoraufruf [43]

- Konstruktoren können sich über `this()` gegenseitig aufrufen
- `this()` muss in der ersten Zeile stehen
- nur ein `this()` pro Konstruktor
- `this()` ist nur in Konstruktoren erlaubt

```
class Kfz {
    int leistung;
    int vmax;
    double gewicht;

    Kfz(int leistung, int vmax, double gewicht) {
        this.leistung = leistung;
        this.vmax = vmax;
        this.gewicht = gewicht;
    }

    Kfz(int leistung, int vmax) {
        this(leistung, vmax, 1.5);
    }

    Kfz(int leistung) {
        this(leistung, 200, 1.5);
    }
}
```



```
Kfz porsche = new Kfz(350, 289, 1.455);

Kfz m6 = new Kfz(507, 305);
System.out.println(m6.gewicht); // -> 1.5

Kfz smart = new Kfz(45);
System.out.println(smart.vmax); // -> 200
System.out.println(smart.gewicht); // -> 1.5
```

Damit man keine Arbeit bei der Initialisierung doppelt machen muss, besteht die Möglichkeit, dass sich Konstruktoren gegenseitig aufrufen. Anders als bei Methoden erfolgt dieser gegenseitige Aufruf allerdings nicht über den Namen, sondern über das Schlüsselwort `this()`. Es ist wichtig zu verstehen, dass es sich bei `this()` um ein Schlüsselwort handelt, dass einen anderen Konstruktor aufruft, bei `this` aber um eine Objektreferenz. Man muss diese beiden sehr Unterschiedlichen Bedeutungen von `this` sauber auseinanderhalten, um nicht durcheinanderzukommen. Obwohl sie sich nämlich im Quelltext sehr ähnlich sehen, ist die Bedeutung für das Programm eine ganz andere.

3.8 Beispiel: Euro [46]

```
class Euro {

    int cent;

    Euro(int euro, int cent) {
        this.cent = euro * 100 + cent;
    }

    Euro(int cent) {
        this.cent = cent;
    }

    // Fortsetzung nächste Folie
```



```
Euro addiere(Euro betrag) {
    return new Euro(this.cent + betrag.cent);
}

Euro subtrahiere(Euro betrag) {
    return new Euro(this.cent - betrag.cent);
}

String alsString() {
    return (cent / 100) + "," + (cent % 100) + " EUR";
}
```




```
}  
}
```

Das Beispiel zeigt eine typische Klasse, die über mehrere Konstruktoren verfügt. Sie repräsentiert einen Geldbetrag, den man entweder in Euro und Cent oder nur in Cent angeben kann. Intern rechnet die Klasse nur mit Cent, nach außen kann sie aber mit Euro und Cent umgehen. Man sieht hier schon eine typische Eigenschaft von Klasse, nämlich eine strikte Trennung zwischen der internen und externen Darstellung der Daten.

```
Euro cola = new Euro(2, 70);  
Euro mayo = new Euro(30);  
Euro burger = new Euro(3, 59);  
  
Euro summe = cola.addiere(mayo).addiere(burger);  
  
System.out.println(summe.asString());
```



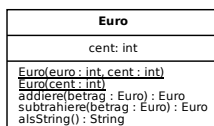
Ausgabe

```
6,59 EUR
```



3.9 Konstruktoren in UML [49]

Konstruktoren werden in UML-Diagrammen unterstrichen



Konstruktoren werden in UML wie Methoden dargestellt, jedoch unterstrichen, um eine Verwechslung zu vermeiden.

Kapitel 4

Sichtbarkeit

4.1 Sichtbarkeit [51]

- *Methodenparameter*: nur innerhalb der Methode
- *Lokale Variablen*: nur innerhalb der Methode
- *Attribute*: überall sichtbar (?)
- *Methoden*: überall sichtbar (?)

In den bisherigen Ausführungen haben wir gesehen, dass es zumindest zwei Stufen der Sichtbarkeit gibt: Lokale Variablen und Methodenparameter sind nur innerhalb der Methode selbst sichtbar, Attribute und Methoden sind von überall aus erreichbar – zumindest ist das unser aktueller Wissensstand.

Hier stellt sich sofort die Frage, ob man mit diesen beiden Stufen der sinnvoll programmieren kann oder ob man nicht noch mehr Einfluss auf die Sichtbarkeit benötigt.

4.2 Fehlende Kapselung [52]

```
class Datum {  
    // Alle Variablen sind von außen zugänglich  
    int tag;  
    int monat;  
    int jahr;  
}
```



```
Datum d = new Datum();  
d.tag = 32; // ungültiger Wert für Tag  
  
// Korrekter Bereich der Werte, aber ungültiges Datum  
d.tag = 30; d.monat = 2;  
  
// Datum läuft über  
d.tag = 31;
```



```
d.tag++;
```

Datum verheimlicht seine Implementierung nicht, sondern erlaubt dem Verwender den direkten Zugriff auf die Instanz-Variablen.

Das Problem hierbei ist, dass es damit zum einen unmöglich ist, jemals den Typ der Variablen zu ändern, z. B. das Datum intern durch ein `java.util.Date` abzubilden oder für `tag` ein Feld vom Typ `byte` zu verwenden. Weiterhin ist es Datum nicht möglich, Konsistenzprüfungen auf den Daten durchzuführen, da beim Zugriff auf eine Variable kein Code ausgeführt wird. Wie im Beispiel zu sehen, können so ungültige Zustände von Datum erzeugt werden, was dazu führt, dass die Verwender von Datum sich nicht darauf verlassen können, dass das Datumsobjekt gültige Informationen trägt. Daher müssen die Verwender selber das Datum auf Richtigkeit prüfen, was eine weitere Verletzung der Kapselung darstellt, da jetzt Programmteile außerhalb des abstrakten Datentyps Datum über dessen Konsistenz wachen müssen.

4.3 Zugriffsbeschränkung [53]

Attribute und Methoden können mit **Zugriffsbeschränkungen** (`access modifier`) versehen werden

Zugriffsbeschränkungen

- `private` – Zugriff nur aus der eigenen Klasse
- `public` – jeder darf zugreifen
- `protected` – Zugriff nur für Subklassen (siehe unten)
- `default` (fehlen einer Angabe) – Zugriff nur aus demselben Paket (siehe unten)

In Java besteht die Möglichkeit, die Sichtbarkeit von Methoden und Attribute in Klassen durch das Voranstellen eines sogenannten `access modifiers` zu kontrollieren. Hierbei stehen vier Stufen zur Verfügung.

4.4 Beispiel mit Beschränkungen [54]

```
class Datum {  
  
    private int tag;  
    private int monat;  
    private int jahr;  
  
    public void setTag(int tag) {  
  
        if (tag > 31) {  
            tag = 31;  
        }  
  
        this.tag = tag;  
    }  
}
```



```
...  
}
```

Natürlich ist der hier gezeigte Source-Code naiv und nicht wirklich gut implementiert. Der Test ob es sich bei dem Tag um eine gültige Möglichkeit handelt würde in der Realität sicherlich anders implementiert werden.

Durch die Kapselung der Felder von `Datum` hat man nun die Möglichkeit, die interne Darstellung durch eine andere zu ersetzen. Z. B. könnte man statt drei einzelner `int`-Werte auch das Konzept des Unix-Timestamps verwenden und einfach die Millisekunden seit einem bestimmten Zeitpunkt zur Bestimmung des Datums verwenden.

```
Datum d = new Datum();  
d.setTag(32); // ungültiger Wert für Tag wird abgefangen
```



4.5 Zugriffsbeschränkung und UML [56]

Zugriffsbeschränkung für Methoden und Attribute wird in UML mit speziellen Zeichen notiert

- + entspricht `public`
- - entspricht `private`
- # entspricht `protected`
- ~ entspricht `default`

In UML wird die Zugriffsbeschränkung von Attributen und Methoden durch voranstellen eines Zeichens markiert.

```
public class Auto {  
    private int vmax;  
    private int v;  
  
    public void beschleunigen() {  
        // ...  
    }  
}
```



Auto
-vmax : int -v : int
+beschleunigen()

Kapitel 5

Vererbung

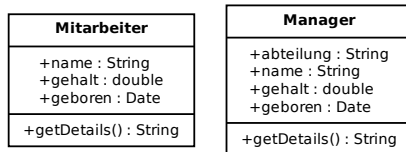
5.1 Motivation [59]

```
public class Mitarbeiter {  
    public String name;  
    public double gehalt;  
    public Date geboren;  
  
    public String getDetails() {  
        return name + ", geb. am " + geboren +  
            " hat ein Gehalt von " + gehalt;  
    }  
}
```



```
public class Manager {  
    public String abteilung;  
    public String name;  
    public double gehalt;  
    public Date geboren;  
  
    public String getDetails() {  
        return name + ", geb. am " + geboren +  
            " hat ein Gehalt von " + gehalt;  
    }  
}
```

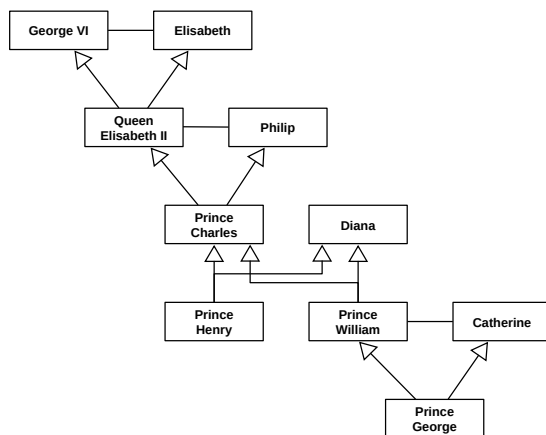




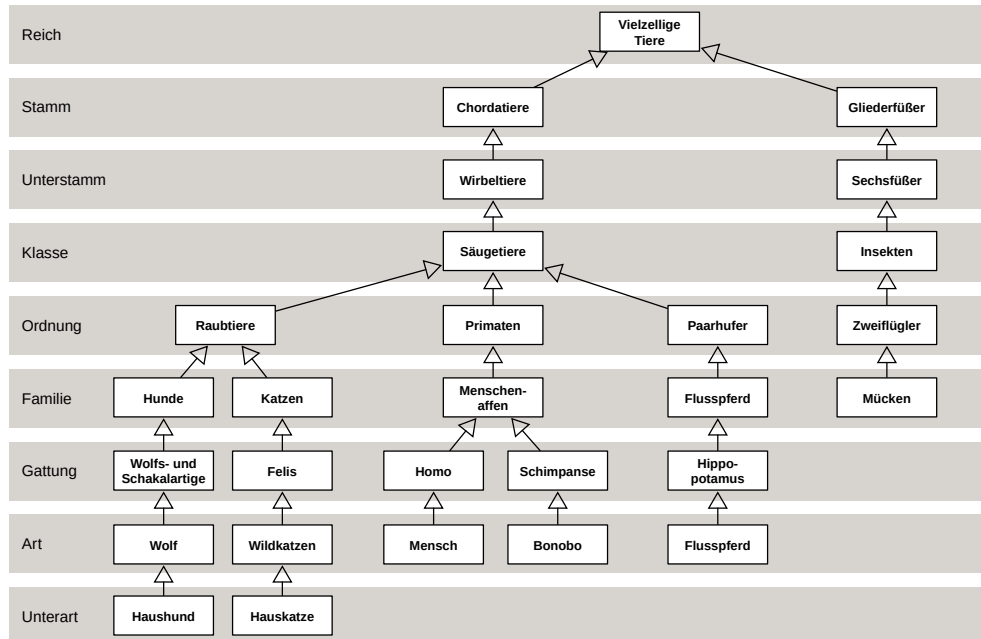
Man sieht an dem Beispiel, dass sehr viel Code dupliziert ist. Obwohl sich die beiden Klassen sehr ähnlich sind gibt es einen kleinen Unterschied: Der Manager leitet eine Abteilung (Attribut `abteilung`). Alle anderen Attribute hat er aber mit dem Mitarbeiter gemeinsam.

5.2 The Royal Baby [62]

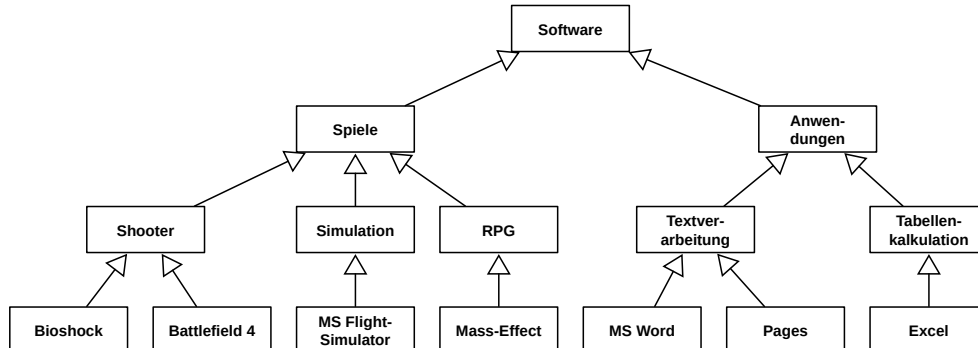
Prince George Alexander Louis of Cambridge (* 22. Juli 2013)



5.3 Biologische Taxonomie [65]



5.4 Programm-Kategorien [66]



5.5 Grundidee der Vererbung [67]

Vererbung (subclassing)

- Klassen können Eigenschaften an andere Klassen übertragen
- **Basisklasse** (base class) überträgt (vererbt)
 - Attribute
 - Methoden
- an die **Abgeleitete Klasse** (derived class), diese definiert

Vererbung

Basisklasse

Abgeleitete Klasse

- ▶ Konstruktoren
- ▶ zusätzliche Attribute
- ▶ zusätzliche Methoden
- Konstruktoren werden *nicht* vererbt

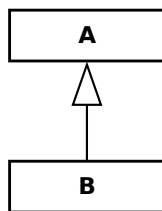
Die Vererbungsbeziehung ist transitiv, d. h. sie bezieht sich nicht nur auf eine direkte Oberklasse, sondern geht auch über mehrere Hierarchieebenen hinweg. Mit jeder weiteren Ableitung erbt die Subklasse alle bereits an ihre Oberklasse vererbten Attribute und Methoden.

Wenn die Anzahl der Vererbungsebenen zu groß wird, kann die Verständlichkeit des Programms deutlich leiden. Daher sollte man vorsichtig mit Vererbungsbeziehungen umgehen, die 6 oder mehr Ebenen haben.

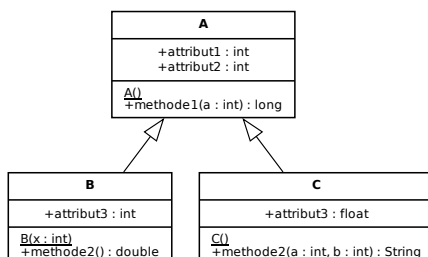
5.6 Vererbung und UML [68]

- Die Vererbungsbeziehung wird in UML durch einen speziellen Pfeil ausgedrückt
- Der Pfeil zeigt von der Abgeleiteten Klasse zur Basisklasse

Beispiel B erbt von A



Die geerbten Methoden und Attribute werden in der Subklasse nicht noch einmal angegeben



5.7 Begriffe [70]

- **Basisklasse** wird auch genannt
 - ▶ Superklasse (**superclass**)
 - ▶ Oberklasse (**superclass**)
 - ▶ Elternklasse (**parent class**)
- **Abgeleitete** Klasse nennt man auch
 - ▶ Subklasse (**subclass**)
 - ▶ Unterklasse (**subclass**)
 - ▶ Kindklasse (**child class**)

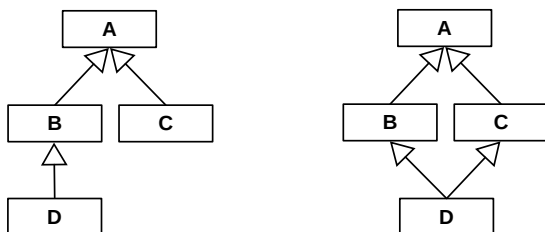
5.8 Einfachvererbung [71]

In Java gibt es grundsätzlich nur **Einfachvererbung** zwischen Klassen

- **Einfachvererbung** (**single inheritance**): Jede Klasse erbt von maximal einer anderen Klassen
- **Mehrfachvererbung** (**multiple inheritance**): Jede Klasse kann von beliebig vielen anderen Klassen erben

Einfachvererbung

Mehrfachvererbung



Einfach- und Mehrfachvererbung

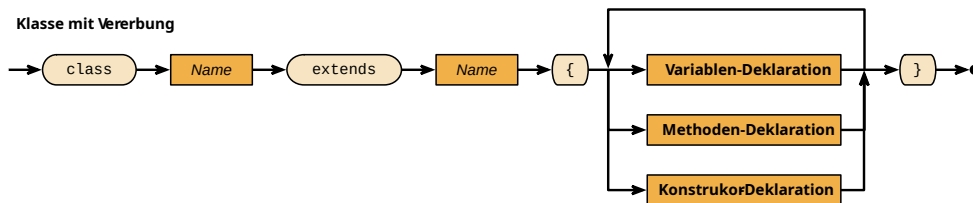
Man hat sich in Java bewusst dazu entschlossen auf Mehrfachvererbung zu verzichten, da sie zu erheblichen Problemen führen kann. Die Sprache C++, die Mehrfachvererbung unterstützt ist berühmt/berüchtigt für die Probleme die entstehen, wenn eine Klasse gleichnamige Attribute und Methoden von mehreren Eltern erbt.

Da es allerdings ein sehr häufiger Anwendungsfall von Mehrfachvererbung in der Beschreibung von Schnittstellen (**Interfaces**) liegt, hat man für diesen Fall in Java ein gesondertes Konstrukt ersonnen, das später behandelt wird. So ist es gelungen, dass man auch ohne Mehrfachvererbung sehr gut objektorientiert mit Java programmieren kann.

Neben der Fähigkeit Daten und Methoden zu Klassen bzw. Objekten zusammenzufassen, stellt Vererbung ein weiteres wichtiges Merkmal objektorientierter Sprachen dar. Fehlt die Vererbung spricht man von objektbasierten Sprachen (z. B. ist Visual Basic 6 nur objektbasiert). Man spricht auch häufig von Ableiten oder Ableitung, wenn man Vererbung meint. Man leitet dann die Klasse B von der Klasse A ab.

5.9 Syntax [72]

- Die Subklasse gibt mit Hilfe des Schlüsselwortes `extends` an, dass sie von einer anderen Klasse erbt
- Syntax: `public class B extends A { ... }`



Syntax einer Klassendeklaration mit Vererbung

5.10 Zurück zum Beispiel [73]

```

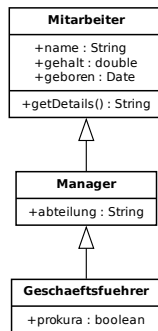
public class Mitarbeiter {
    public String name;
    public double gehalt;
    public Date geboren;

    public String getDetails() {
        return name + ", geb. am " + geboren +
            " hat ein Gehalt von " + gehalt;
    }
}

public class Manager extends Mitarbeiter {
    public String abteilung;
}

public class Geschaeftsfuehrer extends Manager {
    public boolean prokura = true;
}
  
```





5.11 Generalisierung [75]

Generalisierung: aus mehreren Klassen gemeinsame Basisklasse abstrahieren

Generalisierung

- Gemeinsamkeiten in Basisklasse verlagern
- In abgeleiteten Klassen nur noch Spezialitäten
- Jede Instanz der abgeleiteten Klasse zugleich Instanz der Basisklasse
- Beispiele
 - ▶ Auto ist Basisklasse für Sportwagen und Kleinwagen
 - ▶ Mitarbeiter ist Basisklasse für Manager

5.12 Spezialisierung [76]

Spezialisierung: speziellere Klasse aus allgemeiner Basisklasse ableiten

Spezialisierung

- Allgemeine Eigenschaften aus der Basisklasse werden durch die Subklasse verfeinert
- Jede Instanz der abgeleiteten Klasse zugleich Instanz der Basisklasse
- Beispiele
 - ▶ Von Auto kann man auch Cabriolet ableiten
 - ▶ Von Mitarbeiter kann man Pförtner ableiten

5.13 Wann verwendet man Vererbung? [77]

Vererbung modelliert eine *ist eine* Beziehung



Voraussetzung, um Vererbung einzusetzen

- abgeleitete Klasse ist Spezialisierung der Basisklasse
 - ▶ ein Cabriolet ist ein spezielle Art von Auto (ohne Dach)

- ▶ ein Manager ist Mitarbeiter mit mehr Befugnissen
- jede Instanz der abgeleiteten Klasse zugleich Instanz der Basisklasse
 - ▶ wo ein Auto fahren kann, kann auch ein Cabriolet fahren
 - ▶ ein Manager wird genauso verwaltet, wie ein Mitarbeiter

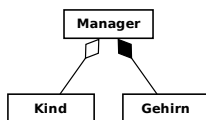
5.14 Wann verwendet man Vererbung nicht? [78]

Besonders wichtig ist die Abgrenzung zwischen Vererbung und

- **Aggregation** modellieren eine „hat“ Beziehung zwischen Klassen: „ein Manager hat Kinder“
- **Komposition** ist eine Aggregation bei denen die Elemente nicht getrennt existieren können: „ein Manager hat ein Gehirn“

Aggregation

Komposition



5.15 Vererbung und Zugriffskontrolle [79]

Zusätzlich zu **public** und **private** gibt es eine weitere Zugriffsstufe Stufe, **protected**

- **private** – Zugriff nur aus der eigenen Klasse
- **public** – jeder darf zugreifen
- **protected** – Zugriff nur für Subklassen
- default (fehlen einer Angabe) – Zugriff nur aus demselben Paket (siehe unten)

```

public class Mitarbeiter {
    protected String name;
    protected double gehalt;
    protected Date geboren;
}

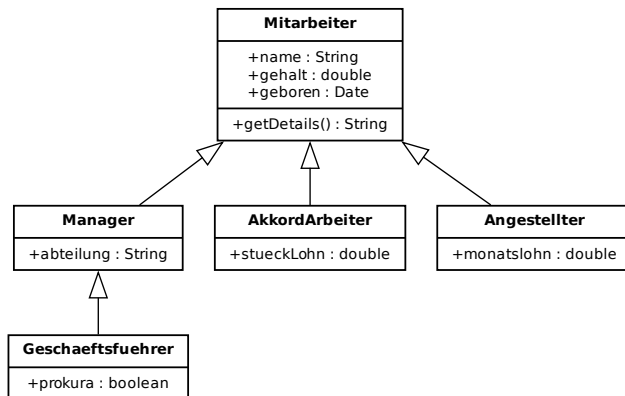
public class Manager extends Mitarbeiter {
    protected String abteilung;

    public String getDetails() {
        return name + ", geb. am " + geboren +
            " hat ein Gehalt von " + gehalt;
    }
}
  
```



5.16 Vererbung über mehrere Ebenen [81]

Vererbungshierarchien können sich über beliebig viele Ebenen erstrecken



5.17 Vererbung und Konstruktoren [82]

Konstruktor der *Basisklasse*

- initialisiert Attribute der Basisklasse
- darf einen anderen Konstruktor der eigenen Klasse mit `this(...)` aufrufen (muss die erste Anweisung sein)

Konstruktor der *abgeleiteten Klasse*

- ruft als erste Anweisung entweder
 - einen anderen Konstruktor der eigenen Klasse mit `this(...)` auf
 - oder einen Konstruktor der Basisklasse mit `super(...)` auf
- falls weder `this(...)` noch `super(...)` aufgerufen wird generiert der Compiler ein `super()` in die erste Zeile des Konstruktors
- Gibt es in der Elternklasse keinen Default-Konstruktor bekommt man einen Compiler-Fehler
- Sind alle Konstruktoren einer Klasse private, kann man von ihr nicht ableiten und von außen auch keine Objekte von ihr erzeugen

5.18 Aufrufhierarchie von Konstruktoren [83]

- Die Aufrufe der Konstruktoren der Superklassen erfolgen bevor der eigene Konstruktor ausgeführt wird, da `this()` und `super()` als erste Zeile im Konstruktor stehen müssen

- Die geerbten Variablen werden vor den eigenen Variablen initialisiert und bevor eigene Anweisungen durchgeführt werden können

Die Konstruktoren der Basisklasse werden **vor** den Konstruktoren der Subklassen ausgeführt. Damit wird sichergestellt, dass die Attribute der Basisklasse zuerst initialisiert werden.



Ein verbreiteter Denkfehler ist es anzunehmen, dass durch die Vererbung bei der Erzeugung eines Objekts der Subklasse Objekte der Superklasse angelegt werden. Dem ist nicht so. Tatsächlich wird nur ein einziges Objekt, nämlich vom Typ der Subklasse, im Speicher allokiert. Dieses enthält Speicherplatz für seine eigenen und alle geerbten Variablen. Die komplexe Aufrufhierarchie der Konstruktoren dient nur dazu, diese Variablen korrekt zu initialisieren, d. h. den Code auszuführen, der zu den Variablen gehört.

5.19 Methoden überschreiben [84]

- Häufig will eine Subklasse nicht exakt das Verhalten der Superklasse erben
- Subklassen können daher Methoden der Superklasse durch eigenen Implementierungen ersetzen, **überschreiben von Methoden** (method overriding)
- Die *überschriebene Methode* hat
 - ▶ denselben Namen
 - ▶ denselben Rückgabotyp
 - ▶ dieselbe Parameterliste
 - ▶ aber möglicherweise ein anderes Verhalten als die Methode aus der Superklasse

überschreiben von
Methoden

5.20 Beispiel: Methoden überschreiben [85]

```
public class Mitarbeiter {  
  
    public String name;  
    public double gehalt;  
    public Date geboren;  
  
    public String getDetails() {  
        return name + ", geb. am " + geboren +  
            " hat ein Gehalt von " + gehalt;  
    }  
}
```



```
public class Manager extends Mitarbeiter {  
    public String abteilung;  
  
    public String getDetails() {
```



```
        return name + ", geb. am " + geburtsdatum +  
            " hat ein Gehalt von " + gehalt +  
            " und leitet die Abteilung " + abteilung;  
    }  
}
```

```
public class Geschaeftsfuehrer extends Manager {  
    public boolean prokura = true;  
  
    public String getDetails() {  
        return name + ", geb. am " + geburtsdatum +  
            " hat ein Gehalt von " + gehalt +  
            " und leitet die Abteilung " + abteilung +  
            " und hat Prokura " + prokura;  
    }  
}
```



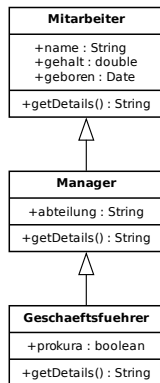
5.21 Aufruf der ursprünglichen Methode [88]

Die ursprüngliche Methode kann mit **super** aus der überschriebenen Methode heraus aufgerufen werden

```
public class Manager extends Mitarbeiter {  
    public String abteilung;  
  
    public String getDetails() {  
        return super.getDetails() +  
            " und leitet die Abteilung " + abteilung;  
    }  
}  
  
public class Geschaeftsfuehrer extends Manager {  
    public boolean prokura = true;  
  
    public String getDetails() {  
        return super.getDetails() + " und hat Prokura " + prokura;  
    }  
}
```



5.22 Überschriebene Methoden in UML [89]



5.23 Tipp: @Override verwenden [90]

- Überschriebene Methode mit `@Override` annotieren
- Der Compiler stellt dann sicher, dass auch wirklich eine Methode aus der Oberklasse überschrieben wird

```
class A {
    public void method(Object o) { /* .. */ }
}

class B extends A {

    @Override
    public void method(String o) { // Fehler!!!
        // ...
    }
}
```



5.24 Finale Klassen [91]

Finale Klasse (final class)

Finale Klasse

- Von finalen Klassen kann nicht geerbt werden
- Syntax: `public final class NAME { ... }`
- Stellen ein Blatt der Vererbungshierarchie dar



```
public final class A { }  
  
public class B extends A { // Fehler!!  
  
}
```

Kapitel 6

Object

6.1 Die Klasse Object [93]

- Die Klasse `Object` ist die Wurzel aller Vererbungshierarchien
- Jede Java-Klasse erbt direkt oder indirekt von `Object`
- Wenn eine Klasse von keiner anderen erbt, erzeugt der Compiler automatisch ein `extends Object`, d. h. aus `public class Klasse { }` wird `public class Klasse  $\leftarrow$  extends Object { }`
- `Object` enthält einige Methoden, die es damit an alle anderen Klassen vererbt. Die wichtigsten sind
 - ▶ `equals()`
 - ▶ `hashCode()`
 - ▶ `toString()`
 - ▶ `clone()`

6.2 toString-Methode [94]

- Liefert eine Darstellung des Objekts als String zurück
- Wird automatisch bei String-Verknüpfungen aufgerufen
- Soll primär beim Debuggen helfen
- Dient nicht der Serialisierung von Objekten
- Die Standardimplementierung gibt den Typ und den Hash-Code aus

```
System.out.println(m          + " hat ein Gehalt von " + gehalt);  
System.out.println(m.toString() + " hat ein Gehalt von "  
    + Double.toString(gehalt));
```



```
@Override  
public String toString() {  
    return "Mitarbeiter [name: " + name + ", "  
    + " Gehalt: " + gehalt + "]";  
}
```



```
+ "gehalt: " + gehalt + ", "  
+ "geburtsdatum " + geburtsdatum + "]" ;  
}
```

6.3 hashCode-Methode [96]

Wenn man equals() überschreibt, so muss man auch hashCode() überschreiben und umgekehrt



- hashCode() dient dazu, einen möglichst eindeutigen Hash-Wert für das Objekt zu erzeugen
- Die Implementierung von Object verwendet einen einmaligen Schlüssel für den Hash-Wert, unabhängig von den Daten im Objekt
- Man überschreibt hashCode(), um einen Hash-Wert aus den Daten des Objekts zu berechnen

Möglichst eindeutig bedeutet, dass es vorkommen kann, dass zwei Objekte mit unterschiedlichem Inhalt den selben Hash-Wert haben. Dies ist nicht zu vermeiden, da der Hash-Wert nur ein einfacher 32bit int Wert ist und daher nicht immer eindeutig sein kann. Man spricht in dem Fall, dass zwei unterschiedliche Objekte den selben Hash-Wert haben von einer hash collision oder einer Kollision der Hash-Werte. Solche hash collisions führen nicht zu Fehlverhalten des Programms sind aber trotzdem aus Gründen der Effizienz zu minimieren. Um hash collisions zu vermeiden muss man versuchen, die erzeugten Hash-Wert möglichst breit über den verfügbaren Wertebereich eines 32bit Integers zu streuen und dafür zu sorgen, dass ähnliche Daten nicht unbedingt ähnliche Hash-Werte bekommen.

Verwendet man die Standard-Implementierung von hashCode() aus Object, so haben zwei unterschiedliche Objekte niemals den selben Hash-Code, auch wenn ihr Inhalt vollkommen gleich ist. Die Virtual Machine greift hierzu auf die Speicherstelle des Objektes zurück, die eindeutig sein muss.

6.4 equals-Methode [97]

Wenn man equals() überschreibt, so muss man auch hashCode() überschreiben und umgekehrt



- Der == Operator bestimmt ob zwei Objekt-Referenzen identisch sind (auf dasselbe Objekt zeigen)
- equals() stellt fest ob zwei Objekte gleich sind (den gleichen Inhalt haben)
- Wenn man equals() nicht überschreibt, verwendet die Implementierung aus Object den == Operator

6.5 Äquivalenzbeziehung [98]

Für `equals()` stellt eine Äquivalenzbeziehung dar

Es gilt (`a`, `b` und `c` sind Objektreferenzen)

- *Symmetrie*
`a.equals(b) == b.equals(a)`
- *Reflexivität*
`a.equals(a) == true`
- *Transitivität*
`a.equals(b) == true` und `b.equals(c) == true`
 $\Rightarrow$ `a.equals(c) == true`
- *Leeres Element*
`a.equals(null) == false`

6.6 Beziehung `equals()` zu `hashCode()` [99]

Wenn für zwei Objekte `a` und `b` gilt

- `a.equals(b) == true`
 $\Rightarrow$ `a.hashCode() == b.hashCode()`
- `a.hashCode() != b.hashCode()`
 $\Rightarrow$ `a.equals(b) == false`

Beachten Sie, dass man nicht aus `a.hashCode() == b.hashCode()` schließen darf, dass `a.equals(b) == true` gilt. Dies wäre ein logischer Fehlschluss und schon deshalb falsch, weil Hash-Codes kollidieren können und dürfen.

Sowohl `equals()` als auch `hashCode()` müssen die selben Felder des Objektes einbeziehen, da andernfalls die obige Bedingung nicht für alle möglichen Wertekombinationen der Felder gewährleistet werden kann.

Kapitel 7

Statische Attribute und Methoden

7.1 Klassenattribute [101]

Klassenattribute (class attributes)

Klassenattribute

- sind nicht dem Objekt, sondern *der Klasse zugeordnet*
- alle Instanzen dieser Klasse teilen sich diesen Wert
- Änderungen wirken sich auf alle Instanzen der Klasse aus
- Klassenattribute als Zustand der Klasse interpretierbar
- Zugriff erfolgt ohne eine konkrete Instanz

Synonym: **statische Attribute** (static attributes)

statische Attribute

7.2 Syntax: Klassenattribute [102]

Klassenattribute

- mit Schlüsselwort **static** gekennzeichnet
- Konstanten werden grundsätzlich als **static** gekennzeichnet
`public static final String WAEHRUNG = "Euro";`

Namen müssen eindeutig sein

- keine zwei Attribute mit gleichem Namen
- keine zwei Klassenattribute mit gleichem Namen
- kein Klassen- und Attribut mit gleichem Namen

7.3 Beispiel: Klassenattribute [103]

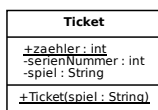
```
public class Ticket {  
  
    public static int zaehler = 0;  
  
    private int serienNummer;  
    private String spiel;  
}
```



```
public Ticket(String spiel) {
    zaehler++;
    serienNummer = zaehler;
    this.spiel = spiel;
}
}
```

7.4 Klassenattribute in UML [104]

Klassenattribute werden in UML unterstrichen



7.5 Zugriff auf Klassenattribute [105]

- innerhalb der Klasse
 - über den Namen (wie bei normalen Attributen)
 - Beispiel: (1 + mwstSatz)
- außerhalb der Klasse
 - über `Klassenname.Klassenattributname`
 - Beispiel: (1 + SteuerSaetze.mwstSatz)

Schlechter Programmierstil: Zugriff über eine Referenzvariable oder `this`

7.6 Klassenmethoden [106]

Klassenmethoden (class methods)

Klassenmethoden

- sind nicht dem Objekt, sondern *der Klasse zugeordnet*
- operieren auf den Klassenattributen
- Zugriff erfolgt ohne eine konkrete Instanz: `Klassenname.methode()`

Synonym: **statische Attribute** (static attributes)

statische Attribute

7.7 Beispiel: Klassenmethode [107]

```
public class Ticket {
```



```

private static int zaehler = 0;

private int serienNummer;
private String spiel;

public Ticket(String spiel) {
    zaehler++;
    serienNummer = zaehler;
    this.spiel = spiel;
}

public static int getZaehler() {
    return zaehler;
}
}

```

```

public class TestTicket {

    public static void main(String[] args) {

        Ticket t1 = new Ticket("Dortmund-Schalke");
        System.out.println("Zähler = " + Ticket.getZaehler());

        Ticket t2 = new Ticket("Hoffenheim-Leverkusen");
        System.out.println("Zähler = " + Ticket.getZaehler());
    }
}

```

```

Zähler = 1
Zähler = 2

```

7.8 Klassenmethoden in UML [110]

Klassenmethoden werden in UML unterstrichen

Ticket
-zaehler : int -serienNummer : int -spiel : String
+Ticket(spiel : String) <u>getZaehler() : int</u>

7.9 Zugriff auf Klassenmethoden [111]

- innerhalb der Klasse
 - ▶ über den Namen (wie bei normalen Methoden)
 - ▶ Beispiel: `getZaehler()`
- außerhalb der Klasse
 - ▶ über `Klassenname.Klassenattributname`
 - ▶ Beispiel: `Ticket.getZaehler()`

Schlechter Programmierstil: Zugriff über eine Referenzvariable oder `this`

7.10 Beschränkungen in Klassenmethoden [112]

Klassenmethoden

- kein Zugriff auf Attribute
- kein Aufruf von Methoden
- keine Nutzung von `this`

Grund: es gibt kein Objekt sondern nur die Klasse

Index

Abgeleitete Klasse, 27
Aggregation, 32
Attribute, 5

Basisklasse, 27

Default Constructor, 16
Default-Wert, 7

Eigenschaften, 3
Einfachvererbung, 29

Finale Klasse, 36
Fähigkeiten, 3

Generalisierung, 31

Identität, 9
Instanzen, 7
Instanziierung, 7
Instanzmethoden, 9

Klasse, 5
Klassenattribute, 41
Klassenextension, 7
Klassenmethoden, 42
Komposition, 32
Konstruktor, 15

Mehrfachvererbung, 29
Methoden, 5

Nachrichten, 3

Objekt, 3, 4
Objektorientierung, 2

Smalltalk, 2
Spezialisierung, 31
statische Attribute, 41, 42

this-Referenz, 11
Typ, 8

verdecken, 12, 13
Vererbung, 27

Zugriffsbeschränkungen, 22

überschreiben von Methoden, 34